

CHAPTER 08: SOFTWARE TESTING [mid 2]

PROGRAM TESTING

- **Testing** ensures the program **works as intended**
 - Helps **find defects (errors)** before deployment
 - Done by **executing** the **program** with **test (artificial) data**
 - Results are checked for:
 - Errors
 - Unexpected behavior (anomalies)
 - Performance and other non-functional aspects
 - **✗ Cannot prove** that the **program is completely error-free**
 - Testing is part of **Verification and Validation (V&V)**
 - Also includes **static techniques** (e.g., code reviews, inspections)
-

PROGRAM TESTING GOALS

♦ 1. Requirement Satisfaction

- **Show that software meets requirements**

✓ For **custom** software:

- At least **one test per requirement**

✓ For **generic** software:

- Test **all features** and **feature combinations**
-

♦ 2. Defect Detection

- Find situations where software:
 - **Behaves incorrectly**
 - **Produces wrong results**
 - **Crashes or corrupts data**
-

VALIDATION AND DEFECT TESTING

♦ 1. Validation Testing

- **Ensures** system **behaves as expected**
 - Uses **normal/expected test cases**
 - Confirms requirements are met
-

♦ 2. Defect Testing

- Aims to **find bugs and faults**
 - Uses **unusual or extreme test cases**
 - Does NOT need to reflect real usage
-

TESTING PROCESS GOALS

♦ Validation Testing Goal

- **Prove system works correctly**
-  Successful test = system behaves as expected

♦ Defect Testing Goal

- **Expose errors in the system**
- ✓ Successful test = system fails and reveals a defect

✓ VERIFICATION vs VALIDATION

♦ Verification

- **"Are we building the product right?"**
- Checks if software **follows** specifications
- Focus on **correct implementation**

♦ Validation

- **"Are we building the right product?"**
- Checks if software **meets** user needs
- Focus on **user requirements**

🎯 Key Difference

Aspect	Verification	Validation
Focus	Specifications	User needs
Question	Product built right?	Right product built?
Type	Static (mostly)	Dynamic (mostly)

✓ V & V CONFIDENCE

- Goal: Ensure system is **fit for purpose**
- Level of confidence depends on:

♦ 1. Software Purpose

- **More critical system** → **higher confidence needed**
 - Example: Banking system vs simple app
-

♦ 2. User Expectations

- Some systems → users expect **less perfection**
 - Example: minor bugs tolerated in games
-

♦ 3. Marketing Environment

- Sometimes **early release is prioritized**
 - Trade-off between:
 - **Speed**
 - **Thorough testing**
-



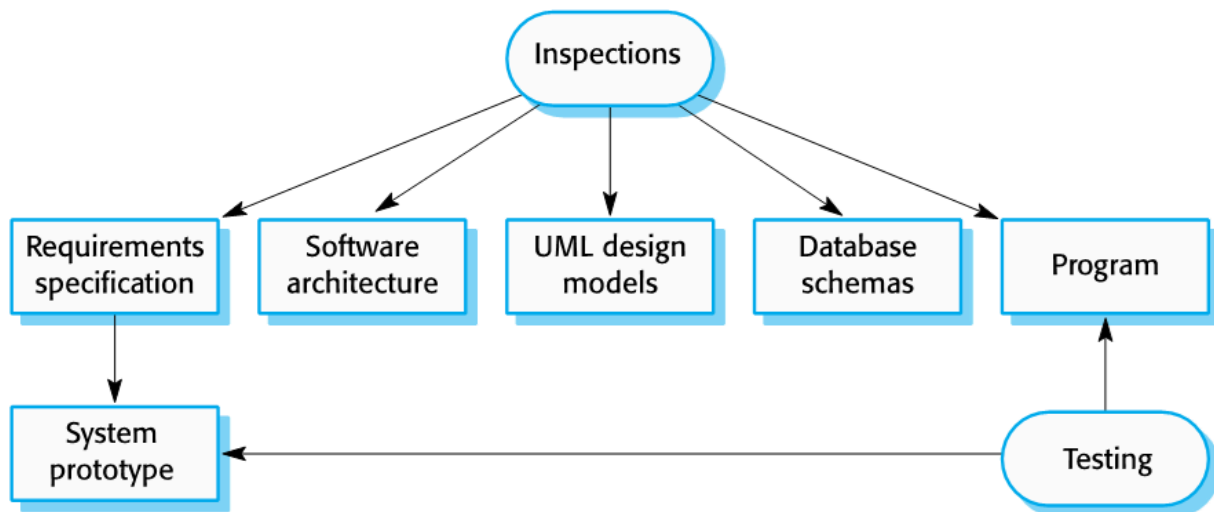
INSPECTIONS vs TESTING

♦ Software Inspections (**Static Verification**)

- **Analyze system** **without executing it**
- Focus on:
 - **Code**
 - **Documents**
- Can use **tools for analysis**

♦ Software Testing (Dynamic Verification)

- Execute the program with **test data**
- **Observe actual behavior/output**



SOFTWARE INSPECTIONS

- Involve **manual examination** of system representations
- Goal: **Find defects and anomalies**
- Do **not require program execution** (static technique)
- Can be used **before implementation** !!!!!!!!!

♦ Where Inspections Can Be Applied

- **Requirements documents**
- **Design documents**
- **Source code**

- Configuration data
 - Test data
-

◆ Key Benefit

- Proven to be effective in detecting errors early
-



ADVANTAGES OF INSPECTIONS

◆ 1. Errors Don't Hide Each Other

- In testing, one error can mask another
 - In inspections (static), this problem is avoided
-

◆ 2. Works on Incomplete Systems

- Can inspect even if system is not fully developed
-

◆ 3. Improves Overall Quality

- Not only finds bugs but also checks:
 - Standards compliance
 - Portability
 - Maintainability
-

✓ INSPECTIONS vs TESTING

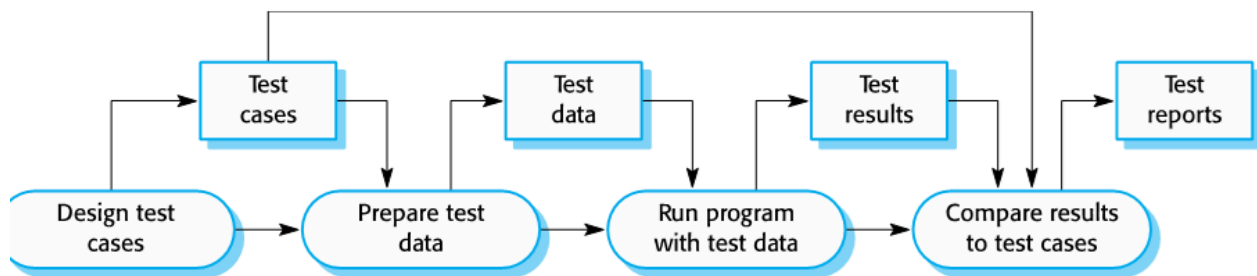
- ✓ Both are **complementary** (work together)
 - ✓ **Both** should be used in **V & V process**
-

♦ What Inspections Can Do

- Check if system **follows specifications**
-

♦ What Inspections Cannot Do

- ✗ **Cannot** ensure system meets real user needs
- ✗ Cannot **test non-functional aspects** like:
 - Performance
 - Usability



✓ STAGES OF TESTING

♦ 1. Development Testing

- Done during **system development**
 - **Performed** by the **development team**
 - Goal: **find and fix bugs early**
-

♦ 2. Release Testing

- Done by a **separate testing team**
 - Tests the **complete system before release**
 - Ensures system is ready for users
-

♦ 3. User Testing

- Done by **real users or potential users**
 - Performed in **real environment**
 - Checks if system **meets user expectations**
-



DEVELOPMENT TESTING TYPES

♦ 1. Unit Testing

- Tests **individual units/components**
 - Focus on **functions or methods**
 - Done in **isolation**
 - Type: **Defect testing**
-

♦ 2. Component Testing

- Combines **multiple units** into **composite components**

- Tests **component interfaces**
 - Test compatibility and proper data transfer
-

♦ 3. System Testing

- Tests **entire system as a whole**
 - Focus on **interaction between components**
-



UNIT TESTING

- Tests **smallest parts of the system**
- Done **independently** from other components
- **Defect** testing process

♦ Units Can Be:

- **Functions** or **methods**
 - Object **classes**
 - **Composite components** with interfaces
-



OBJECT CLASS TESTING

♦ To fully test a class:

- Test **all operations (methods)**
 - Check all **attributes (variables)**
 - Test object in **all possible states**
-

♦ **Challenge:**

- **Inheritance** makes testing **harder**
 - Because behavior is **spread across** parent and child classes
-

WEATHER STATION TESTING **(EXAMPLE)**

WeatherStation
identifier
reportWeather () reportStatus () powerSave (instruments) remoteControl (commands) reconfigure (commands) restart (instruments) shutdown (instruments)

♦ **Functions to Test:**

- reportWeather
- calibrate
- test
- startup
- Shutdown

♦ **State-Based Testing:**

- **Identify** state transitions
 - **Test sequences of events**
-

♦ **Example Test Sequences:**

- Shutdown → Running → Shutdown
- Configuring → Running → Testing → Transmitting → Running

- Running → Collecting → Running → Summarizing → Transmitting → Running

AUTOMATED TESTING

- **Unit testing** should be **automated whenever possible**
- Tests run **without manual intervention**
- Uses **test automation frameworks** (e.g., JUnit)
- Frameworks:
 - Provide **predefined test classes**
 - **Allow** creation of **custom test cases**
 - Run all tests and **report results (often via GUI)**

AUTOMATED TEST COMPONENTS

♦ 1. Setup Part

- **Initialize test conditions**
 - Define:
 - Inputs
 - Expected outputs
-

♦ 2. Call Part

- **Execute** the **method/function being tested**
-

♦ 3. Assertion Part

- **Compare actual result** with **expected result**
-  True → Test passes

-  False → Test fails
-



CHOOSING UNIT TEST CASES

♦ Goal

- Ensure component works correctly
 - Reveal any existing defects
-

♦ Types of Test Cases

✓ 1. Normal Test Cases

- Based on **expected usage**
- **Show system works correctly**

✓ 2. Abnormal Test Cases

- Use **invalid or extreme inputs**
 - Check system:
 - **Handles errors properly**
 - **Does not crash**
-



TESTING STRATEGIES

♦ 1. Partition Testing (equivalence testing)

- **Divide inputs** into **groups (partitions)**

- Each **group** has **similar behavior**
 - **Select test cases from each group**
-

♦ 2. Guideline-Based Testing

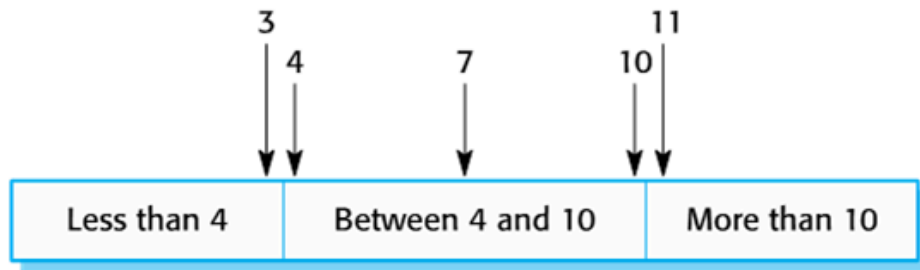
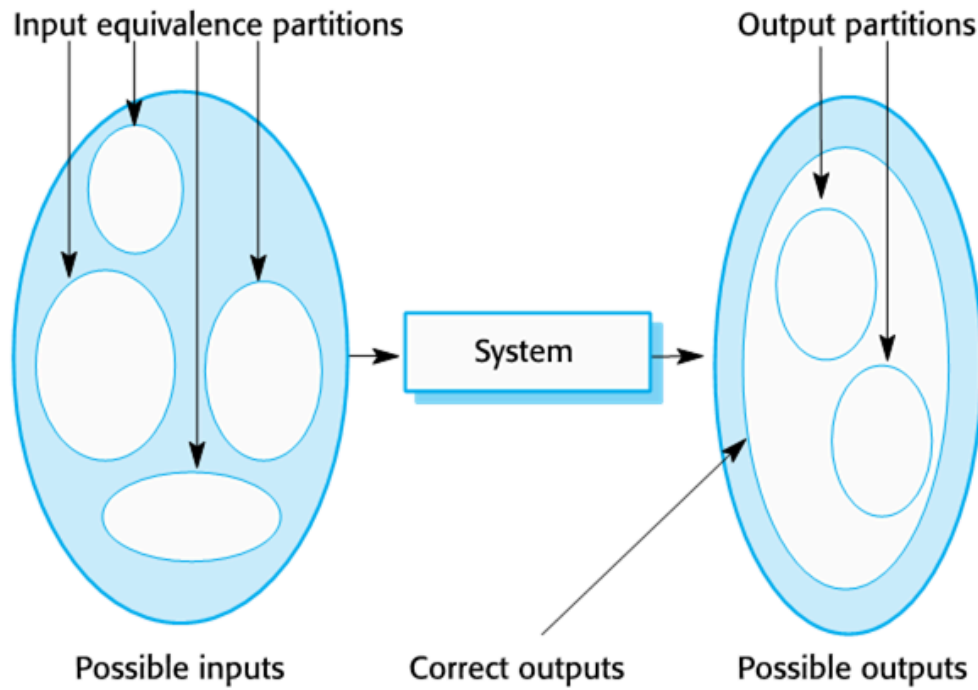
- Based on **past experience**
 - Focus on **common programming errors**
-

PARTITION TESTING

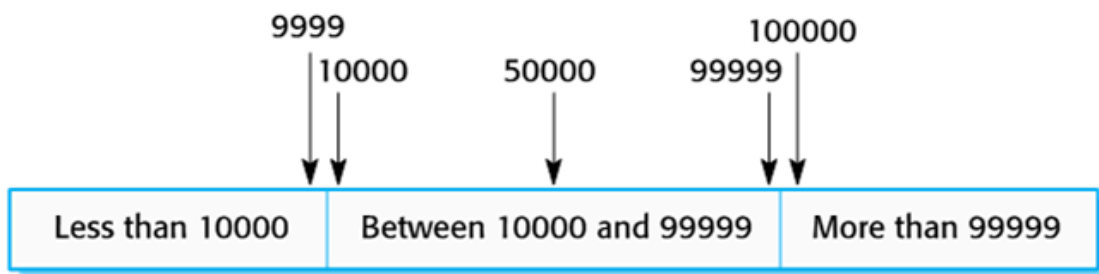
- Inputs/outputs are divided into **equivalence classes**
 - Each class behaves **similarly**
-

♦ Key Idea

- **Test one or more values from each partition**
- Ensures **coverage without testing all inputs**



Number of input values



Input values



TESTING GUIDELINES (SEQUENCES)

- **Test sequences** with **only one element**
 - Some programs fail when there's only **one item**
 - Use sequences of **different sizes**
 - check if program works for **all sizes**
 - Test access to:
 - **First element**
 - **Middle element**
 - **Last element**
 - Test with **empty sequences (zero length)**
 - Many programs crash when there is **no data**
-



GENERAL TESTING GUIDELINES

- **Choose inputs** that **force** system **to generate all error messages**
 - **Design inputs** to cause **buffer overflow**
 - Repeat same inputs **multiple times**
 - **Force system** to produce **invalid outputs**
 - Test with **values** that are:
 - **Too large**
 - **Too small**
-



COMPONENT TESTING

- Components are often made of **multiple interacting objects**
- Focus on testing the **component as a whole**

◆ Key Points

- **Access** component via its **defined interface**
 - Ensure interface behaves **as specified**
 - **Assume unit testing of individual objects is already done**
-

✓ INTERFACE TESTING

Interface Testing

Testing components that enable interaction between different components.
Interface errors can't be identified from unit testing.

Interface types: Parameter interfaces, Shared memory interfaces, Procedural interfaces, Message passing interfaces.

Types of interface errors: Interface misuse (wrong parameters), Interface misunderstanding (wrong assumptions), Timing errors (shared memory speed mismatch).

◆ Types of Interfaces

✓ 1. Parameter Interfaces

- **Data passed** between **methods/functions**
-

✓ 2. Shared Memory Interfaces

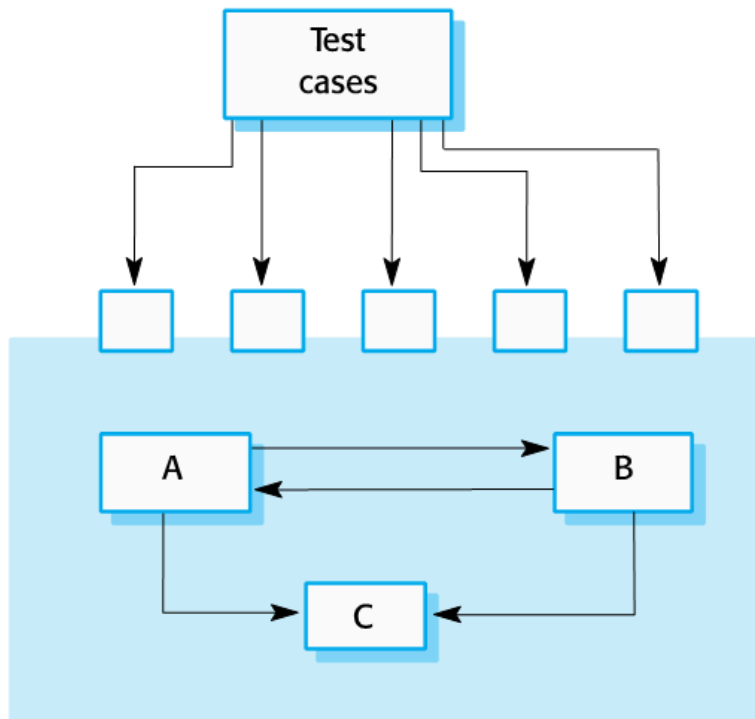
- Multiple **components share** the **same memory space**
-

✓ 3. Procedural Interfaces

- **Subsystem** provides **procedures/functions to others**
-

✓ 4. Message Passing Interfaces

- Subsystems **communicate** by **sending messages/requests**



✓ INTERFACE ERRORS

♦ 1. Interface Misuse

- **Calling component uses interface incorrectly**
 - Example: **Passing parameters** in **wrong order**
-

♦ 2. Interface Misunderstanding

- **Wrong assumptions** about how another component works
 - Example: Assuming a function returns sorted data when it doesn't
-

♦ 3. Timing Errors

- Components run at **different speeds**
 - Leads to:
 - Using **outdated data**
 - **Synchronization issues**
-



INTERFACE TESTING GUIDELINES

- Test parameters at **extreme values (min/max)**
 - Test pointer parameters with **null values**
 - Create tests that **force failures**
 - Use **stress testing** in message-passing systems
 - In shared memory systems:
 - Change **execution order of components**
-



SYSTEM TESTING

- **Combine components** → test **complete system**
- Focus on:
 - **Component interactions**
 - **Data flow** between components

♦ Checks in System Testing

- Components are **compatible**
- **Correct data transfer**
- Proper **timing of interactions**

♦ Emergent Behavior

- Behavior that appears only when system is **fully integrated**
- Not visible in individual components



SYSTEM vs COMPONENT TESTING

- System testing:
 - **Tests whole system**
 - **Often involves multiple teams**
- May include:
 - Reusable components
 - Third-party (off-the-shelf) systems

♦ Important Point

- System testing is a **team activity**, not individual
-

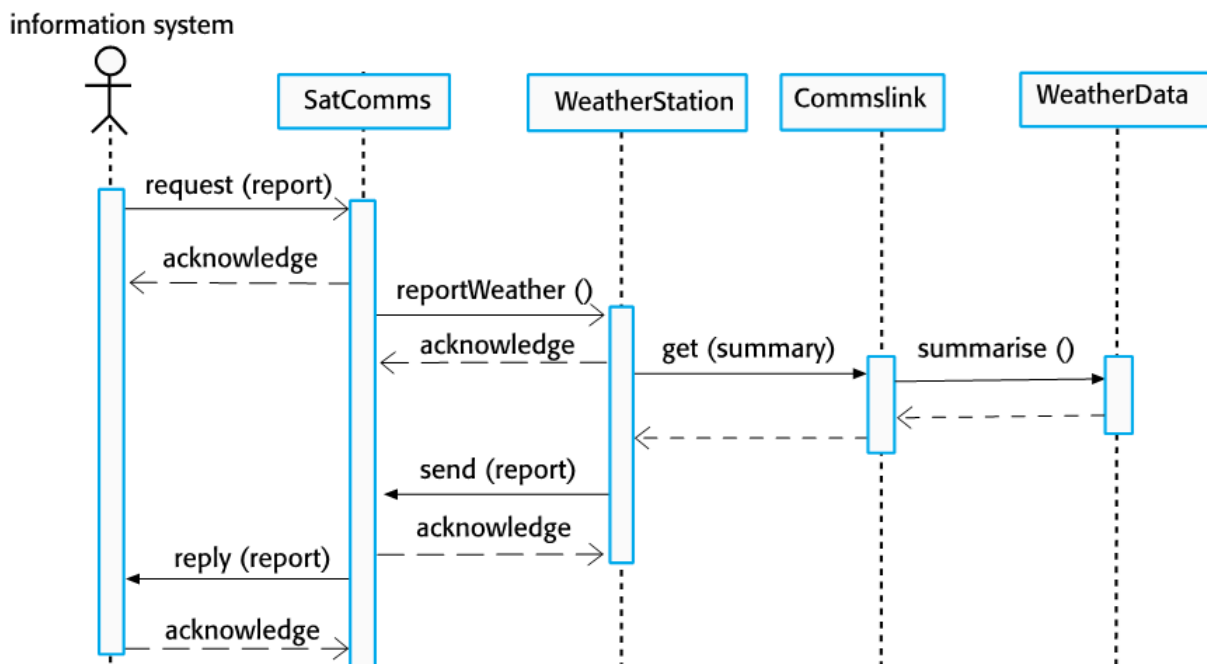
✓ USE-CASE TESTING

- Based on **use cases (user scenarios)**
- Each use case tests:
 - **Multiple components together**
- Use cases force **real interactions** in the system

◆ Sequence Diagrams

- Show:
 - Components involved
 - Interaction flow
- Help design **test cases**

Collect weather data sequence chart



The diagram represents interaction between these components:

- **Information System (user/system)**
- **SatComms**
- **WeatherStation**
- **Commslink**
- **WeatherData**

Flow of events:

1. User sends **request(report)** to *SatComms*
2. SatComms sends **reportWeather()** to *WeatherStation*
3. WeatherStation asks Commslink: **get(summary)**
4. Commslink calls WeatherData: **summarise()**
5. Data flows back → WeatherStation → SatComms → User
6. Final output: **report is returned**

Each step also includes **acknowledgements** (confirmations).

Test Case 1: Request–Acknowledgement Pair

An input of a request for a report should have an associated acknowledgement.

✓ **What to test:**

- When **request is sent** → **system responds** with **acknowledgment**
-

Test Case 2: End-to-End Report Generation

A report should ultimately be returned from the request.

✓ **What to test:**

- **Complete flow works from start to finish**

✓ **Example:**

Input: request(report)

Expected Output: reply(report)

✓ **Test Case 3: Data Organization (Summary Validation)**

Statement: You should create summarized data to check correctness.

✓ **What to test:**

- The report structure is correct
- Summary matches expected format/content

✓ **Example:**

Prepare:

- Known input data
- Expected summary

Then compare:

- System output vs expected summary
-

✓ **Test Case 4: WeatherStation Behavior**

Statement:

Request to WeatherStation results in summarized report.

✓ What to test:

- WeatherStation correctly processes raw data into summary

✓ Example:

Input to WeatherStation: raw weather data

Expected Output: summarized report

✓ Test Case 5: Lower-Level Component Testing

Statement:

Raw data is used to test WeatherStation and WeatherData.

✓ What to test:

- WeatherData → correctly summarizes raw data
 - Commslink → correctly retrieves summary
 - WeatherStation → correctly integrates results
-

1. Testing Policies

Definition

- Testing policies are **rules or guidelines** that define **how much** and **what type of testing** should be done.
 - Since **exhaustive testing** (testing everything) is **impossible**, policies help ensure **adequate coverage**.
-

Key Points

- It is not feasible to test all possible inputs and scenarios.
 - Testing policies help in:
 - Ensuring **important parts of the system are tested**
 - Maintaining **consistency in testing**
 - Defining **minimum required test coverage**
-

Examples of Testing Policies

✓ Menu-Based Testing

- All system functions accessible through menus **must be tested**.
 - Ensures that **user-facing features work properly**.
-

✓ Combination Testing

- Different combinations of functions should be tested.
 - Example:
 - Text formatting options like **bold + italic + underline**
 - Ensures **features work correctly together**, not just individually.
-

✓ Input Validation Testing

- All functions should be tested with:
 -  **Correct input**
 -  **Incorrect/invalid input**
- Helps detect:
 - Errors
 - Crashes
 - Unexpected behavior

2. Test-Driven Development (TDD)

Definition

- **Test-Driven Development (TDD)** is a development approach where:
 - **Tests** are **written before** writing **the actual code**.
 - **Code** is **written only** to **pass the test**.

Key Characteristics

- Testing and coding are done **together (interleaved)**.
- Development is **incremental** (small steps).
- A **feature** is considered **complete** only when:  Its **test passes**

Important Concepts

- Tests act as a **guide for development**.
- Encourages **clean, minimal, and correct code**.
- Common in **Agile** methods like:
 - **Extreme Programming (XP)**

3. TDD Process Activities

Step-by-Step Process

- ✓ **Step 1: Identify Small Functionality**

- Choose a **small feature or increment**
 - Should be:
 - Simple
 - Easy to implement
-

✓ Step 2: Write a Test

- Create an **automated test case**
 - Define:
 - **Expected input**
 - **Expected output**
-

✓ Step 3: Run the Test (It Fails ✗)

- Since code is not written yet:
 - Test will **fail initially**
 - This ensures the test is valid
-

✓ Step 4: Write Code

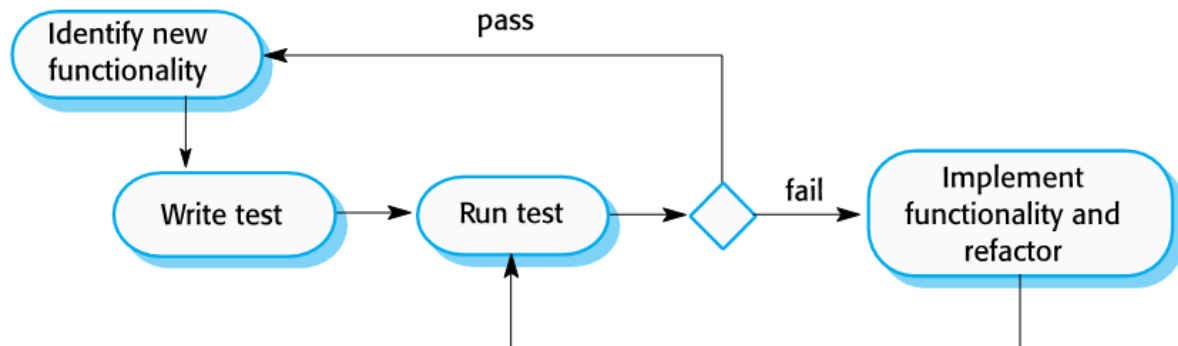
- Implement **just enough code to:**
 - **Pass the test**
-

✓ Step 5: Re-run Tests

- Run:
 - New test
 - All previous tests
- Ensure:
 -  All tests pass

✓ Step 6: Repeat

- Move to next small functionality
- Follow same cycle



◆ 4. Benefits of Test-Driven Development

✓ Code Coverage

- **Every piece of code** has:
 - **At least one test**
- Results in:
 - **High test coverage**
 - Fewer untested parts

✓ Regression Testing

- **Tests are added continuously**
- Creates a **regression test suite**
- Helps ensure:

- **New changes don't break old features**
-

✓ **Simplified Debugging**

- If a **test fails**:
 - The **issue** is likely in **recently added code**
 - Makes debugging:
 - Faster
 - Easier
-

✓ **System Documentation**

- Tests describe:
 - What the system should do
 - Acts as:
 - **Living documentation**
 - Helpful for:
 - Developers
 - Future maintenance
-

1. Regression Testing

Definition

- Regression testing is done to **ensure that new changes do not break existing functionality.**
 - It checks that **previously working code still works correctly** after updates.
-

Key Points

- Performed after:
 - Bug fixes
 - Code modifications
 - Feature additions
 - **Focus** is on **already tested features**.
-

Manual vs Automated Regression Testing

✓ **Manual** Testing

- **Time-consuming** and **expensive**
- Requires repeated human effort

✓ **Automated** Testing

- **Fast** and efficient
 - All tests can be **re-run automatically after every change**
-

Important Rule

- All regression tests must **pass successfully** before:
 - Code is committed
 - Changes are finalized
-

2. Release Testing

Definition

- Release testing is performed on a **complete system version (release)** before it is delivered to users.
 - Ensures the system is **ready for external use**.
 - To convince stakeholders that:
 - The system is **good enough to be released**
 - It **meets** required **quality standards**
-

What Release Testing Checks

- ✓ **Functionality** (does it work as expected?)
 - ✓ **Performance** (is it fast enough?)
 - ✓ **Dependability** (is it reliable and stable?)
 - ✓ **Normal usage behavior** (no failures in everyday use)
-

Testing Type

- Usually **Black-Box Testing**
 - Tests are based on:
 - **System requirements**
 - **Specifications**
 - Internal code is **not considered**
-

3. Release Testing vs System Testing

Similarity

- Release testing is a **type of system testing**.
-

Key Differences

✓ Team Responsibility

- **System Testing:**
 - Done by **development team**
 - **Release Testing:**
 - Done by a **separate independent team**
-

✓ Objective

- **System Testing (Defect Testing):**
 - Goal: **Find bugs and errors**
 - **Release Testing (Validation Testing):**
 - Goal: Ensure system:
 - **Meets requirements**
 - Is **ready for real-world use**
-

4. Requirements-Based Testing

Definition

- Testing approach where:
 - Each **requirement is converted into test cases**
 - Ensures:
 - **All system requirements are properly implemented**
-

Key Idea

- For every requirement:

- Create one or more **test cases**
- Guarantees:
 - **Full requirement coverage**

A usage scenario for the Mentcare system



George is a nurse who specializes in mental healthcare. One of his responsibilities is to visit patients at home to check that their treatment is effective and that they are not suffering from medication side effects.

On a day for home visits, George logs into the Mentcare system and uses it to print his schedule of home visits for that day, along with summary information about the patients to be visited. He requests that the records for these patients be downloaded to his laptop. He is prompted for his key phrase to encrypt the records on the laptop.

One of the patients that he visits is Jim, who is being treated with medication for depression. Jim feels that the medication is helping him but believes that it has the side effect of keeping him awake at night. George looks up Jim's record and is prompted for his key phrase to decrypt the record. He checks the drug prescribed and queries its side effects. Sleeplessness is a known side effect so he notes the problem in Jim's record and suggests that he visits the clinic to have his medication changed. Jim agrees so George enters a prompt to call him when he gets back to the clinic to make an appointment with a physician. George ends the consultation and the system re-encrypts Jim's record.

After, finishing his consultations, George returns to the clinic and uploads the records of patients visited to the database. The system generates a call list for George of those patients who He has to contact for follow-up information and make clinic appointments.

◆ 1. Features Tested by Scenario (Scenario-Based Testing)

📌 Definition

- Scenario testing involves:
 - Creating **real-life usage situations (scenarios)**
 - **Testing** how the **system behaves** in those situations

Features Tested

✓ **Authentication (Login)**

- Test user login functionality
 - Check:
 - Correct username/password → access granted
 - Incorrect credentials → access denied
-

✓ **Uploading & Downloading Patient Records**

- Test transfer of patient data to/from a device (e.g., laptop)
 - Check:
 - Files upload/download correctly
 - No data loss or corruption
-

✓ **Home Visit Scheduling**

- Test scheduling system for appointments
 - Check:
 - Correct date/time assignment
 - Conflict handling (double booking)
-

✓ **Encryption & Decryption**

- Test security of patient records on mobile devices
- Check:
 - Data is properly encrypted
 - Correct decryption when accessed

✓ Record Retrieval & Modification

- Test accessing and updating patient records
- Check:
 - Correct data retrieval
 - Changes are saved properly

✓ Drug Database Integration

- Test link with database containing side effects
- Check:
 - Correct drug information retrieval
 - Accurate side-effect warnings

✓ Call Prompting System

- Test system reminders/prompts (e.g., alerts for actions)
 - Check:
 - Prompts appear at correct time
 - Relevant information is shown
-

2. Performance Testing

Definition

- Performance testing evaluates:
 - **Speed**
 - **Responsiveness**

- **Stability** under different workloads
-

Key Points

- Focuses on **system behavior under load**
 - Tests **emergent properties** like:
 - Performance
 - Reliability
-

Important Concepts

✓ **Realistic Usage Testing**

- Tests should reflect:
 - How the system is actually used in real life
-

✓ **Load Testing**

- Gradually increase system load
 - Observe:
 - When performance starts degrading
-

✓ **Stress Testing**

- **Push** system **beyond limits** (**overload**)
 - Goal:
 - See how system **fails**
 - Check recovery capability
-

3. User Testing

Definition

- Testing performed by:
 - **Real users or customers**
 - Users provide:
 - **Feedback**
 - **Suggestions**
-

Key Points

- Done after system/release testing
 - Important because:
 - **Real-world conditions cannot be fully simulated**
-

Why It Is Important

- Users' environment affects:
 - **Reliability**
 - **Performance**
 - **Usability**
 - **Robustness**
-

4. Types of User Testing

✓ Alpha Testing

- Conducted at:
 - Developer's site
 - **Users work:**
 - Closely with developers
 - Goal:
 - Identify early issues
-

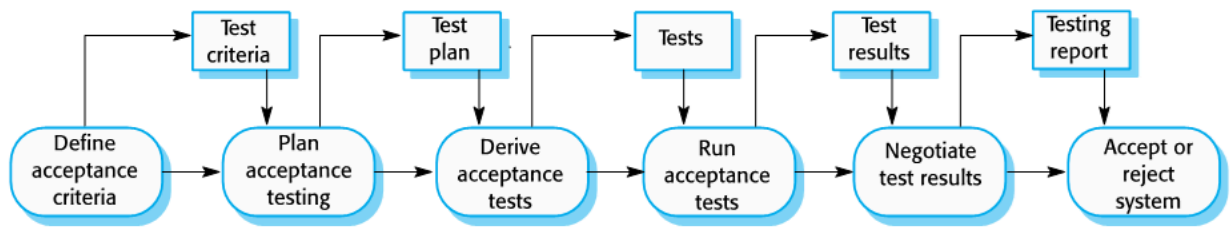
✓ Beta Testing

- **Software released to:**
 - Selected users
 - Users:
 - Test in real environment
 - Report issues
-

✓ Acceptance Testing

- Done by:
 - Customer/client
 - Goal:
 - Decide whether **system is ready for deployment**
-

5. Stages in Acceptance Testing



Step-by-Step Process

- 1) **Define** acceptance criteria
- 2) **Plan** acceptance testing
- 3) **Derive** acceptance tests
- 4) **Run** acceptance tests
- 5) **Negotiate** test results
- 6) **Reject/accept** system

✓ Define Acceptance Criteria

- **Set conditions that system must meet**
-

✓ Plan Acceptance Testing

- Decide:
 - **Testing strategy**
 - **Resources**
-

✓ Derive Acceptance Tests

- **Create test cases** based on:
 - **Requirements**

✓ Run Acceptance Tests

- Execute all test cases

✓ Negotiate Test Results

- Discuss:
 - **Issues found**
 - **Possible fixes**

✓ Accept/Reject System

- Final decision:
 - ✓ **Accept** → **Deploy system**
 - ✗ **Reject** → Fix issues

6. Agile Methods and Acceptance Testing

Key Points

- User/customer is:
 - **Part of development team**
 - Responsible for:
 - **Deciding system acceptability**
-

Characteristics

- Tests are:
 - **Defined by users**
 - **Automated**
 - **Run:**
 - Whenever **changes** are made
-

Important Difference

- No separate acceptance testing phase
 - **Testing** is:
 - **Continuous**
-

Limitation

- Embedded user may:
 - **Not represent all stakeholders**
-

Black box testing validates **software functionality** without knowing the internal code, focusing on user experience, while white box testing examines the **internal structure, code, and logic**

1. Path Testing (Concept)

Definition

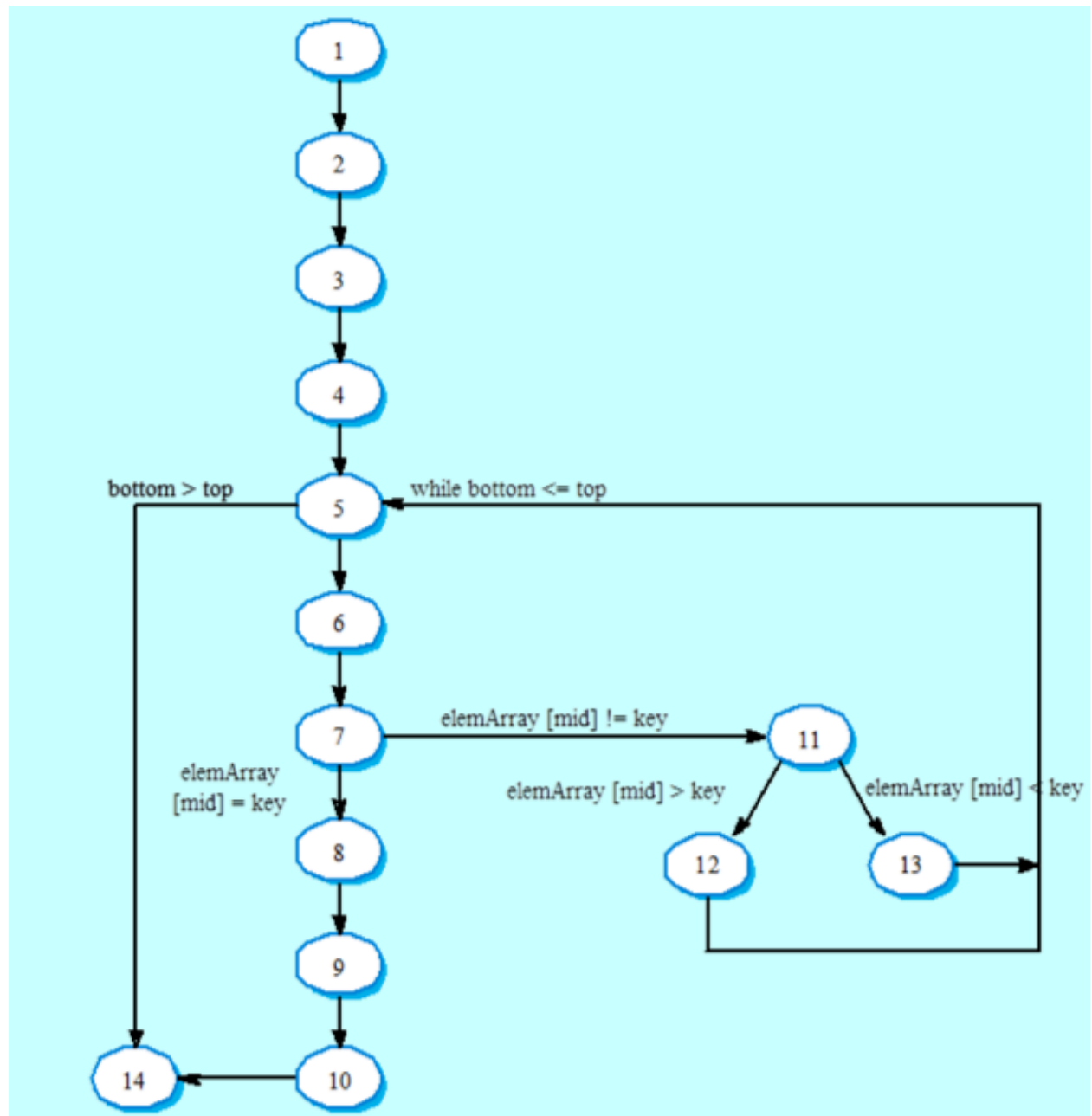
- Path testing is a **white-box testing technique**.
- Objective:

- Ensure **every possible execution path** in a program is **tested at least once**.
-

Key Idea

- A program can execute in **different paths depending on conditions**.
 - Each path represents:
 - A unique sequence of statements
-

2. Program Flow Graph (From Image)



📌 What the Image Shows

- The diagram shows a **control flow graph**:
 - **Nodes** → **program statements** (like 2, 3, 4)
 - **Arrows** → **flow of execution**
-

Important Elements

✓ Nodes

- Represent:
 - **Statements or decision points**
 - Example:
 - Steps $2 \rightarrow 3 \rightarrow 4$
-

✓ Edges (Arrows)

- Represent:
 - **Flow of control** from one statement to another
-

✓ Decision Condition

- In the image:
 - `while (bottom <= top)`
 - This creates:
 - **Loop paths** (repeat execution)
-

Key Insight

- Flow graph helps identify:
 - **All possible execution paths**
 - These paths are used to design:
 - Test cases
-

3. Steps for Basis Path Testing

✓ Step 1: Draw Control Flow Graph

- Convert code into:
 - Nodes (statements)
 - Edges (flow)
-

✓ Step 2: Find Independent Paths

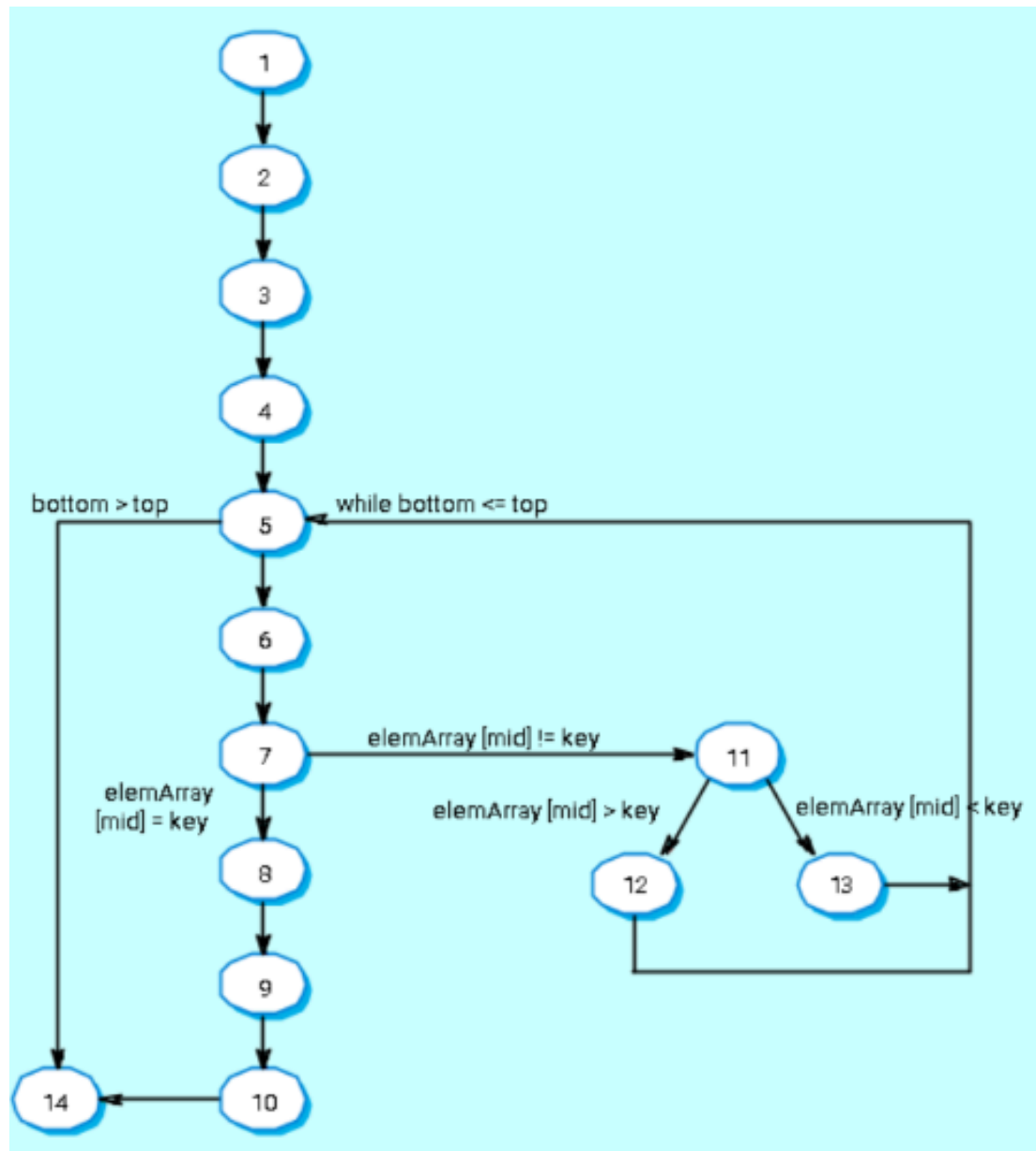
- Identify **unique paths** through the program
 - Each **path introduces**:
 - **At least one new edge**
-

✓ Step 3: Design Test Cases

- Create test inputs such that:
 - **Each path is executed at least once**
-

◆ 4. Code Explanation (Binary Search)

```
public static void search ( int key, int []elemArray, Result r )
{
1. int bottom =0 ;
2. int top =elemArray.length - 1 ; int mid;
3. r.found =false ;
4. r.index =-1 ;
5. while ( bottom <= top )
{
6 mid =(top + bottom) / 2 ;
7 if (elemArray [mid] = key)
{
8 r.index = mid;
9 r.found =true ;
10 return ; } // if part
else
{
11 if (elemArray [mid] < key)
12 bottom = mid + 1 ;
else
13 top = mid - 1 ; }
} //while loop
14. } //search
```



📌 What the Code Does

- This is a **binary search algorithm**
- It searches for a **key** in an array

📌 Main Variables

- `bottom` → start index
 - `top` → end index
 - `mid` → middle index
-

Logic Flow

✓ Initialization

- Set:
 - `bottom = 0`
 - `top = last index`
 - Mark:
 - `found = false`
-

✓ Loop Condition

- `while (bottom <= top)`
 - Keeps searching until:
 - Element is found OR
 - Search space ends
-

✓ Inside Loop

1. Calculate middle:
 - `mid = (top + bottom)/2`
 2. Check if element found:
 - If `elemArray[mid] == key`
 - Found → return result
-

3. Else condition:

- If `elemArray[mid] < key`
 - Search right side → `bottom = mid + 1`
 - Else:
 - Search left side → `top = mid - 1`
-

5. Independent Paths (Given)

These are **different execution paths**:

✓ Path 1 (Element Found Immediately)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 14

- Element found in first check
 - Loop exits early
-

✓ Path 2 (Loop Not Entered)

1 → 2 → 3 → 4 → 5 → 14

- Condition `bottom <= top` is false initially
 - No search performed
-

✓ Path 3 (Search Right Side)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 11 → 12 → 5 → ...

- Element is greater than mid

- Moves to right half
 - Loop repeats
-

✓ Path 4 (Search Left Side)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 11 → 13 → 5 → ...

- Element is smaller than mid
 - Moves to left half
 - Loop repeats
-

6. Test Case Design

Goal

- Create inputs such that:
 - **Each path is executed**
-

✓ Example Test Cases

1. **Element present at middle**
 - Covers Path 1

2. **Empty array or invalid range**
 - Covers Path 2

3. **Element in right half**

- Covers Path 3

4. Element in left half

- Covers Path 4

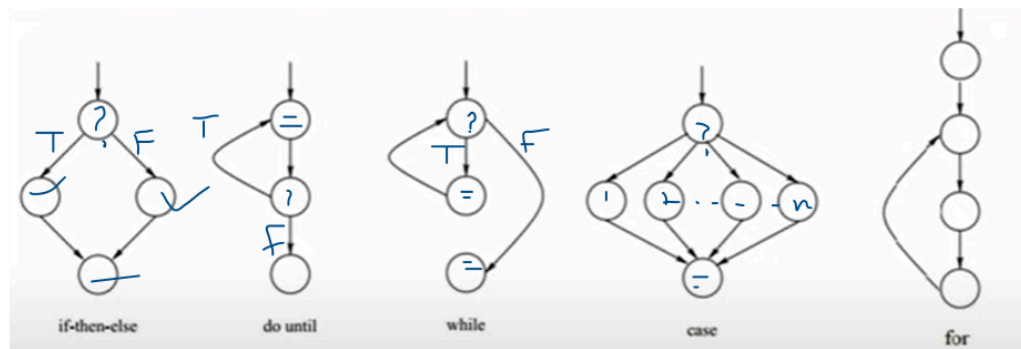
7. Role of Dynamic Program Analyzer

- Tool used to: **Track execution paths during runtime**

Benefit

- **Confirms:**
 - **Whether all paths have been tested**
- Ensures:
 - **Complete path coverage**

Control flow graph symbols & CC formula



✓ 1. If-Then-Else

- A **decision node (?)** splits into two paths:
 - **True (T)** branch
 - **False (F)** branch
- Both paths eventually **merge back**

👉 Meaning:

- Program chooses one of two paths based on condition
-

✓ 2. Do-Until Loop

- Executes the **body first**, then checks condition
- Loop continues **until condition becomes true**

👉 Key Point:

- Executes **at least once**
-

✓ 3. While Loop

- Condition checked **before execution**
- If condition is:
 - True → loop executes
 - False → exits loop

👉 Key Point:

- May execute **zero or more times**
-

✓ 4. Case (Switch Statement)

- One decision node leads to **multiple branches**
- Each branch represents:
 - A different case condition

👉 Key Point:

- Only **one path is executed**
-

✓ 5. For Loop

- Has:
 - Initialization (`int j = 0`)
 - Condition (`j < 5`)
 - Increment (`j++`)
- Behaves like a **while loop**

👉 Key Point:

- Repeats until condition becomes false
-

📦 2. Cyclomatic Complexity (cc)

📌 Definition

- Cyclomatic Complexity **measures**:
 - **Number of independent paths in a program**
-

📌 Formula (from image)

$$CC = E - N + 2P$$

 Where:

- **E** = Number of edges
 - **N** = Number of nodes
 - **P** = Number of connected components (usually = 1)
-

 Simplified Formula (Most Common)

- For a single program:

$$CC = E - N + 2$$

3. Alternative Way to Calculate CC

✓ **Count Decision Points**

- **CC = Number of decision nodes + 1**
-

 Decision Points Include:

- **if**
- **while**

- **for**
 - **case** (each case counts)
-

◆ 4. Example (From For Loop in Image)

```
for (int j = 0; j < 5; j++)
```

✓ Decision:

- Condition: `j < 5`

👉 So:

- 1 decision → CC = 2
-

◆ 5. Why Cyclomatic Complexity is Important

✓ Helps in Testing

- **Minimum number of test cases required** =
👉 **Cyclomatic Complexity**
-

✓ Measures Code Complexity

- **Low CC** → **Simple code**
- **High CC** → **Complex, harder to test**

✓ Improves Quality

- Encourages:
 - Simpler design
 - Better maintainability
-

6. Key Observations from Image

- Every:
 - **Decision** → increases complexity
 - Loops:
 - **Add new execution paths**
 - Case statements:
 - Add multiple paths
-

7. Quick Summary

- CFG shows **program flow visually**
- Nodes = statements, edges = flow
- CC measures: **Number of independent paths**
- Formula:
 - **$CC = E - N + 2P$**
- Also:
 - **$CC = \text{decisions} + 1$**

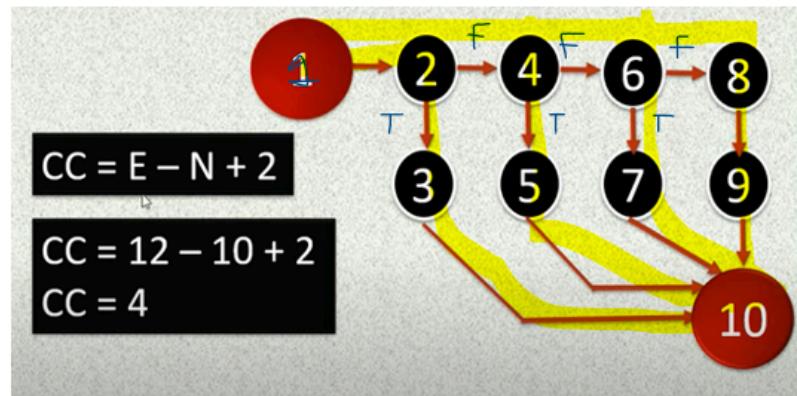
Example 1 -- Step-1: label steps

Step 1:	Input a, b, c // sides > 0
Step 2:	If (a>=(b+c) or b>=(a+c) or c>=(a+b)) Then
Step 3:	Output = "Not a Triangle"
Step 4:	Elseif (a==b and a==c) Then
Step 5:	Output = "Equilateral Triangle"
Step 6:	Elseif (a==b or a==c or b==c) Then
Step 7:	Output = "Isosceles Triangle"
Step 8:	Else
Step 9:	Output = "Scalene Triangle"
Step 10:	Return Output

Step-2: create graph



Step 1:	Input a, b, c // sides > 0
Step 2:	If (a>=(b+c) or b>=(a+c) or c>=(a+b)) Then
Step 3:	Output = "Not a Triangle"
Step 4:	Elseif (a==b and a==c) Then
Step 5:	Output = "Equilateral Triangle"
Step 6:	Elseif (a==b or a==c or b==c) Then
Step 7:	Output = "Isosceles Triangle"
Step 8:	Else
Step 9:	Output = "Scalene Triangle"
Step 10:	Return Output



📌 Cyclomatic Complexity (CC)

Given in image:

$$CC = E - N + 2$$

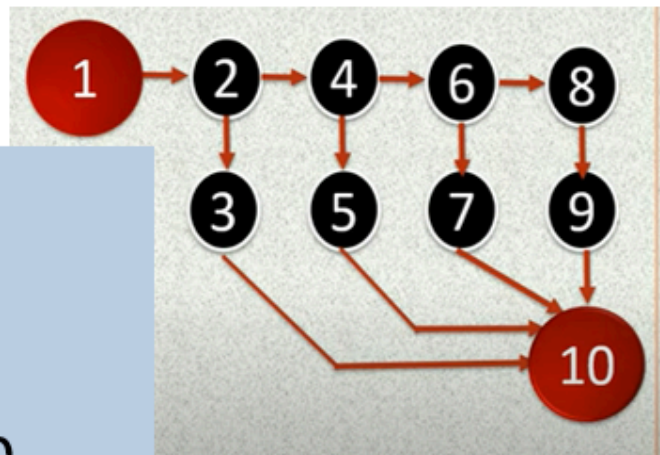
$$CC = 12 - 10 + 2 = 4$$

📌 Meaning of CC = 4

- There are 4 independent paths
- So, we need at least 4 test cases

Step-3: select paths

1-2-3-10
1-2-4-5-10
1-2-4-6-7-10
1-2-4-6-8-9-10



Step-3: create tests

Step 1:	Input a, b, c // sides > 0
Step 2:	If (a>=(b+c) or b>=(a+c) or c>=(a+b)) Then
Step 3:	Output = "Not a Triangle"
Step 4:	Elseif (a==b and a==c) Then
Step 5:	Output = "Equilateral Triangle"
Step 6:	Elseif (a==b or a==c or b==c) Then
Step 7:	Output = "Isosceles Triangle"
Step 8:	Else
Step 9:	Output = "Scalene Triangle"
Step 10:	Return Output



If	Else if	Else if	Else	Expected outcome
T ✓	F	F	F	Not a triangle
F ✗	T ✓	F	F	Equilateral
F	F	T	F	Isosceles
F	F	F	T	Scalene

Example 2: perform basis path testing for following code



```
public double calculate(int amount)
```

```

-1- double rushCharge = 0;
-1- if (nextday.equals("yes") )
{
-2-   rushCharge = 14.50;
}
-3- double tax = amount * .0725;
-3- if (amount >= 1000)
{
-4-   shipcharge = amount * .06 + rushCharge;
}
-5- else if (amount >= 200)
{
-6-   shipcharge = amount * .08 + rushCharge;
}
-7- else if (amount >= 100)
{
-8-   shipcharge = 13.25 + rushCharge;
}

```

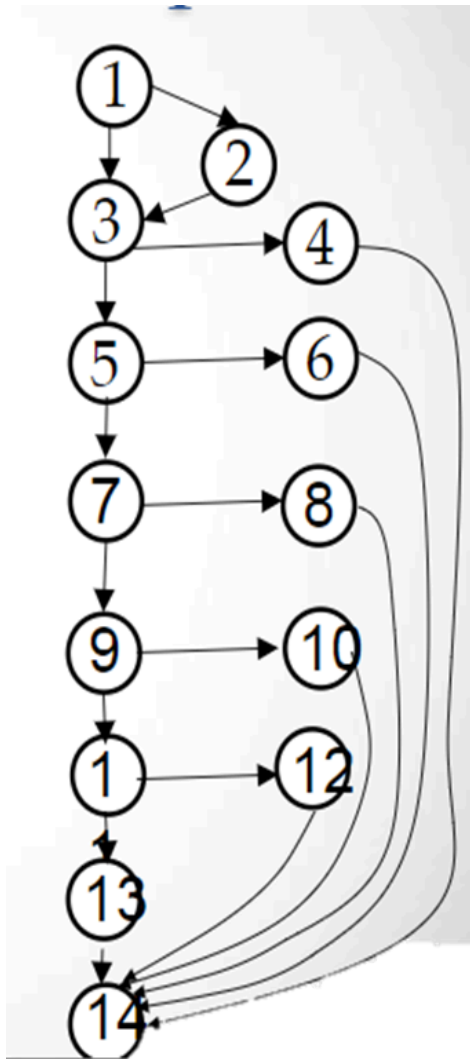
4/9/2026

```

-9- else if (amount >= 50)
{
-10-   shipcharge = 9.95 + rushCharge;
}
-11- else if (amount >= 25)
{
-12-   shipcharge = 7.25 + rushCharge;
}
else
{
-13-   shipcharge = 5.25 + rushCharge;
}
-14- total = amount + tax + shipcharge;
-15- return total;
} //end calculate

```

Chapter 24 Quality management



Step 3: identify paths

P-01: 1, 2, 3, 5, 7, 9, 11, 13, 14

P-02: 1, 3, 4, 14

P-03: 1, 3, 5, 6, 14

P-04: 1, 3, 5, 7, 8, 14

P-05: 1, 3, 5, 7, 9, 10, 14

P-06: 1, 3, 5, 7, 9, 11, 12, 14

P-07: 1, 3, 5, 7, 9, 11, 13, 14

Step 4: define test case

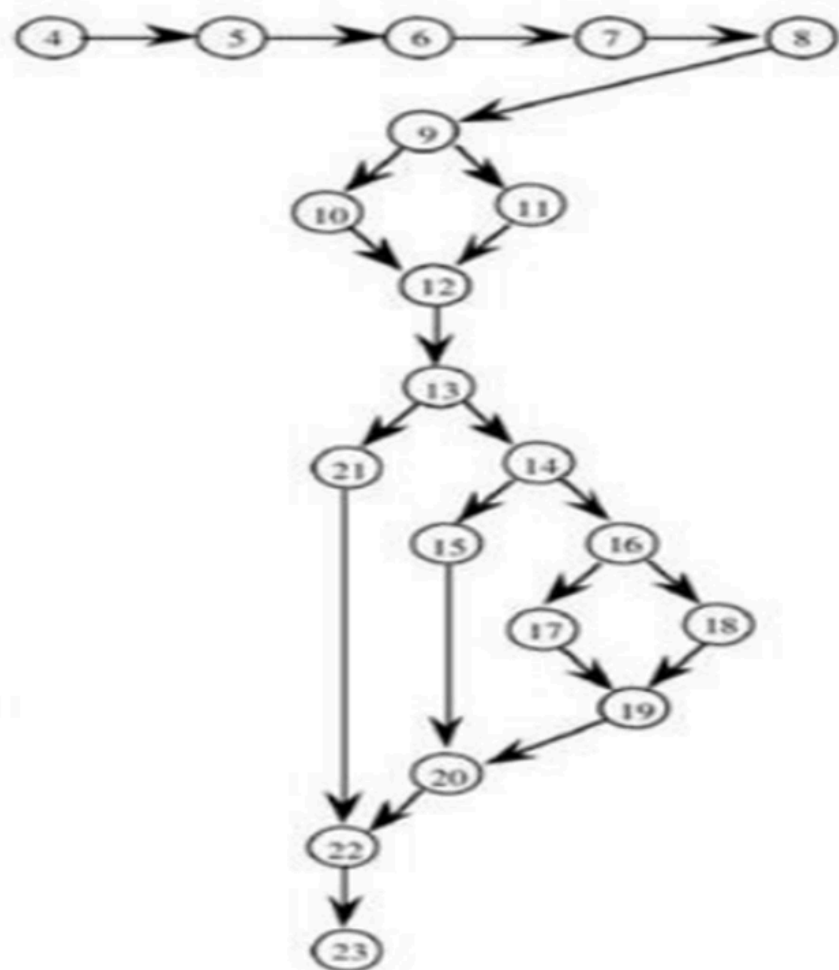
Path #	Amount	NextDay	Expected outcome
2	1500	Yes	? ✓
1	10	Yes	?

Example 2: perform basis path testing for following code



```
1. Program triangle
2. Dim a,b,c As Integer
3. Dim IsATriangle As Boolean
'Step 1: Get Input
4. Output("Enter 3 integers which are sides of a triangle")
5. Input(a,b,c)
6. Output("Side A is ",a)
7. Output("Side B is ",b)
8. Output("Side C is ",c)
'Step 2: Is A Triangle?
9. If (a < b + c) AND (b < a + c) AND (c < a + b)
10.     Then IsATriangle = True
11.     Else IsATriangle = False
12. EndIf

'Step 3: Determine Triangle Type
13. If IsATriangle
14.     Then If (a = b) AND (b = c)
15.         Then Output ("Equilateral")
16.     Else If (a ≠ b) AND (a ≠ c) AND (b ≠ c)
17.         Then Output ("Scalene")
18.     Else Output ("Isosceles")
19.     EndIf
20. EndIf
21. Else Output("Not a Triangle")
22. EndIf
23. End triangle2
```



Example 3: Create control flow graph & calculate Cyclomatic Complexity



```
static void test (Graphics g) throws IOException {  
    int a = 10;  
    int x = System.in.read();  
    int y = System.in.read();  
    if (x < 0)  
        x = 0;  
    if (y < 0)  
        y = 0;  
    while (y <= 210) {  
        g.drawLine(x1: 8, x, a, y);  
        y += 25;  
    }  
}
```

What the Program Does

- Takes inputs **x** and **y**
- If values are negative:
 - Converts them to **0**
- Then:
 - Runs a **loop** while **y <= 210**
 - Increments **y** by 25 each time

2. Decision Points (Very Important)

We identify **conditions (decision nodes)**:

1. **if (x < 0)** → Decision 1
2. **if (y < 0)** → Decision 2

3. `while (y <= 210)` → Decision 3

3. Cyclomatic Complexity (CC)

 Formula (easy method)

$$\text{CC} = \text{Number of decisions} + 1$$

 Calculation

- Decisions = 3
- So:

$$\text{CC} = 3 + 1 = 4 \quad \text{CC} = 3 + 1 = 4 \quad \text{CC} = 3 + 1 = 4$$

 Final Answer

 Cyclomatic Complexity = 4

4. Control Flow Graph (CFG)

Explanation

 Nodes (Steps)

1. Start
2. Initialize variables
3. Check `x < 0`

4. Assign $x = 0$ (if true)
 5. Check $y < 0$
 6. Assign $y = 0$ (if true)
 7. Check `while` ($y \leq 210$)
 8. Loop body (draw + increment)
 9. End
-

Flow Behavior

- Program flows **sequentially**
 - At each **if**, flow splits into:
 - **True** branch
 - **False** branch
 - `while` creates:
 - **Loop back edge**
-

5. Independent Paths (Very Important)

Since $CC = 4$, we need **4 independent paths**:

✓ Path 1: Both conditions FALSE, loop skipped

- $x \geq 0, y > 210$
 - No loop execution
-

✓ Path 2: $x < 0$ TRUE

- x becomes 0
 - Other conditions normal
-

✓ Path 3: $y < 0$ TRUE

- y becomes 0
 - Loop executes
-

✓ Path 4: Loop executes multiple times

- $y \leq 210$ initially
 - Loop runs repeatedly
-

◆ 6. Example Test Cases

Test Case	x	y	Expected Behavior
TC1	5	300	No loop
TC2	-5	300	x becomes 0
TC3	5	-10	y becomes 0 + loop runs
TC4	5	50	Loop executes multiple times

◆ 7. Key Takeaways

- Each **if/while** adds complexity

- CC tells:
 - Minimum number of test cases needed
 - Here:
 - **3 decisions** → **CC = 4** → **4 paths**
-

Exam Tip

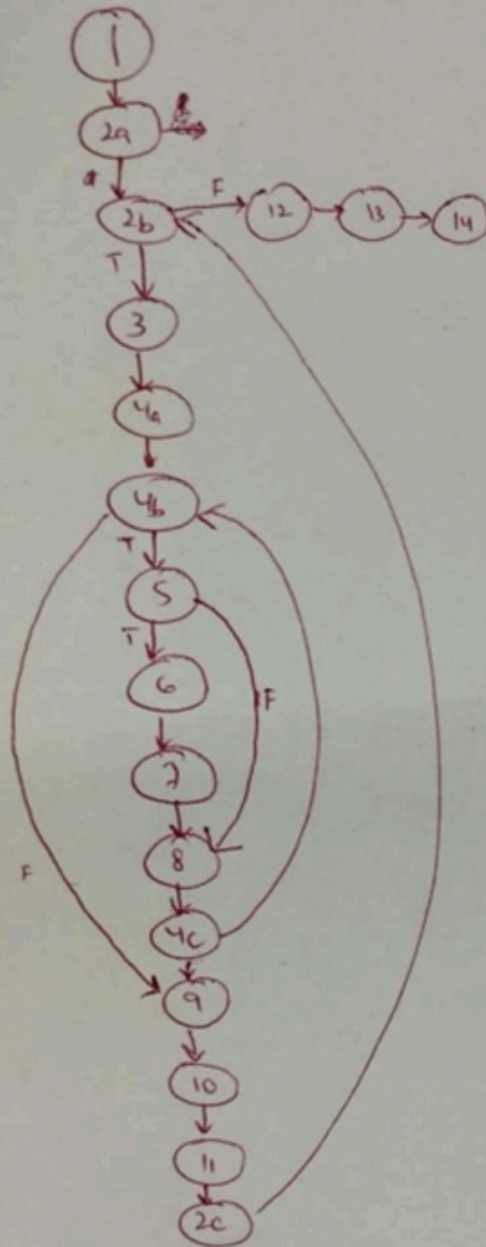
 Steps to solve:

1. Count decision statements
2. Apply:
 - $CC = \text{decisions} + 1$
3. Draw CFG (optional)
4. List independent paths

4: Create control flow graph & identify paths

```
import java.util.Arrays;
class SelectionSort {
    public static void main(String args[]) {
        int[] array = {20, 12, 10, 15, 2};
        for (int step = 0; step < array.length - 1; step++) {
            int min_idx = step;
            for (int i = step + 1; i < array.length; i++) {
                // Select the minimum element in each loop.
                if (array[i] < array[min_idx]) {
                    min_idx = i;
                }
            }
            // put min at the correct position
            int temp = array[step];
            array[step] = array[min_idx];
            array[min_idx] = temp;
        }
        System.out.println("Sorted Array in Ascending Order: ");
        System.out.println(Arrays.toString(array));
    }
}
```

i)



$$ii) CC = E - N + 2$$

$$CC = 21 - 18 + 2$$

$$CC = 5$$

iii) D/Y.

iv)

for 1	for 2	if	exp lines.
F	-	-	13, 14
T	T	F	9, 10, 11, ...
T	T	T	6, 7, ...
T	F	T	9, 10, 11, ...
T	F	F	9, 10, 11, ...

